Foco: O *Como*. Pilha de tecnologia, segurança e engenharia.

- **Frontend Architecture:**
  - *Stack:* React + TypeScript + Vite + Tailwind CSS para estilização utilitária.
  - *Gerenciamento de Estado:* Zustand para dados globais leves (auth, tenant context). React Hook Form + Zod para formulários.
  - *Componentização:* UI baseada em padrões atômicos e reutilizáveis (Inputs, Modais, Tabelas customizadas).
- **Backend & Banco de Dados:**
  - *Estratégia Multi-tenant:* Isolamento de dados por chave estrangeira (`tenant_id`) em nível de banco de dados ou schemas separados para máxima segurança.
  - *Padrão de API:* RESTful estruturado, transporte via JSON, utilizando tokens JWT curtos em conjunto com HTTP-only Cookies para Refresh Tokens.
- **Performance & Motor de Regras:** Estrutura assíncrona baseada em filas (ex: RabbitMQ/Redis) para processar grandes volumes de transações de vendas simultâneas sem travar as requisições HTTP dos usuários.

1. Visão Geral da Stack Tecnológica

O ecossistema é projetado sob o padrão de separação total entre cliente e servidor (*Decoupled Architecture*):

- **Frontend:** Single Page Application (SPA) construída com **React 19**, **TypeScript**, **Vite** (para bundling ultrarrápido) e **Tailwind CSS** (estilização por utilitários e camadas base/componentes).
- **Backend:** API RESTful construída em **Node.js (NestJS)** ou **Python (FastAPI)**, devido à alta performance no processamento assíncrono de E/S.
- **Banco de Dados: PostgreSQL**. A escolha por um banco relacional robusto é mandatória pela necessidade de transações ACID estritas para dados financeiros e suporte nativo ao tipo `NUMERIC` para precisão decimal.

2. Estratégia de Multi-Tenancy (Isolamento de Dados)

Para garantir que a empresa *amazonet* nunca veja os dados de outra empresa inquilina, adotamos a estratégia de **Isolamento Lógico (Shared Database, Shared Schema)** com segurança reforçada:

- **Chave de Particionamento:** Toda e qualquer tabela do banco de dados (exceção feita às tabelas globais do sistema SaaS) possui uma coluna obrigatória `tenant_id: varchar`.
- **Segurança a Nível de Banco (Mecanismo de Defesa):**
  - No Backend, todas as queries SQL geradas pelo ORM obrigatoriamente injetam o filtro `WHERE tenant_id = :current_tenant`.

- Para prevenção de falhas humanas (esquecer o `WHERE`), será implementada a funcionalidade de **Row Level Security (RLS)** nativa do PostgreSQL, bloqueando qualquer leitura que não pertença ao contexto do usuário autenticado na requisição.

3. Modelagem de Dados Resumida (Entidades Críticas)

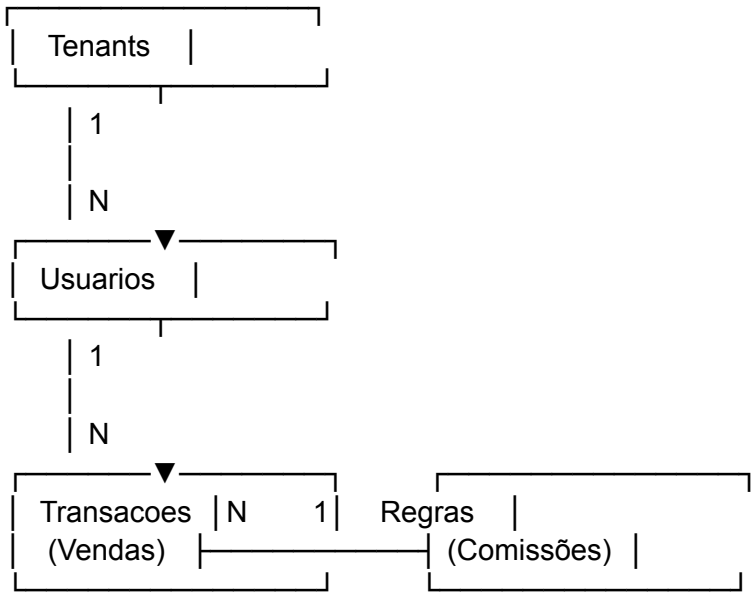


Tabela: `transacoes` (Ingestão de Vendas)

- `id: uuid` (Primary Key)
- `tenant_id: varchar` (Indexed)
- `codigo_externo: varchar` (ID vindo do CRM para idempotência)
- `usuario_id: uuid` (Foreign Key apontando para o beneficiário)
- `valor_bruto_centavos: integer` (Armazenado em centavos para evitar ponto flutuante)
- `valor_liquido_centavos: integer`
- `status: enum` (`PENDENTE`, `PAGO`, `ESTORNADO`)
- `criado_em: timestamp`

4. Arquitetura do Frontend (Estrutura de Pastas)

O frontend segue uma organização modular e escalável voltada a domínios de negócio:

```
src/
├── assets/            # Arquivos estáticos (imagens, logos)
├── components/
│   ├── common/        # Componentes globais isolados (Ex: Logo.tsx)
│   ├── layout/        # Casca da aplicação (Header.tsx, Footer.tsx)
```

```
│   └── ui/             # Elementos atômicos e puros (Input.tsx, Button.tsx)
├── layouts/            # Templates de páginas (DashboardLayout.tsx)
├── pages/              # Telas da aplicação (Login.tsx, Dashboard.tsx)
├── store/              # Gerenciamento de estado global (Zustand - authStore.ts)
├── utils/
│   └── validators/     # Esquemas de validação do Zod (auth.ts)
├── index.css           # Camadas globais e utilitários do Tailwind
└── main.tsx            # Ponto de entrada do React
```

5. Fluxo de Autenticação e Segurança da API

O tráfego de dados sensíveis entre o React e o Servidor utiliza o protocolo **OAuth2 com tokens JWT**:

1. **Handshake Inicial:** O cliente envia `tenantId`, `email` e `password` através do formulário gerenciado pelo *React Hook Form*.
2. **Geração do Token:** O servidor valida os dados, confirma o isolamento do Tenant e retorna duas chaves:
    - `access_token` (JWT assinado com expiração curta de 15 minutos) entregue no corpo da resposta JSON.
    - `refresh_token` (expiração longa de 7 dias) enviado via **HTTP-Only Cookie**, protegido contra ataques XSS.
3. **Requisições Autenticadas:** O frontend armazena o `access_token` em memória volatil (variável de estado ou contexto) e injeta o header `Authorization: Bearer <token>` em todas as chamadas do Axios através de um **Interceptor**.

Com esse quarto documento estruturado, finalizamos o ciclo de fundação da engenharia e produto do **KomissX**. Agora, todo o conhecimento lógico, funcional, visual e de dados está documentado!